Предсказание оттока студентов в LMS

Решение кейса ООО Цифровое образование (Цифриум)

Команда:
Антон Дудецкий
Виктор Морозов
Никита Мачульский
София Канюкова

April 14, 2026

Постановка задачи

Пусть есть множество пользователей

$$\mathcal{U} = \{u_1, u_2, \ldots, u_n\}$$

и множество курсов

$$\mathcal{C} = \{c_1, c_2, \ldots, c_m\}.$$

Объект моделирования:

$$x_i = (u_i, c_i),$$

то есть конкретный пользователь на конкретном курсе.
Для каждого такого объекта строится вектор признаков

$$X_i \in \mathbb{R}^d,$$

который описывает курс, пользователя и его поведение в системе.
Целевая переменная:

$$Y_i = \mathbb{1}(\text{пользователь завершил модуль}).$$

Итоговая задача модели:

$$\hat{p}_i = P(Y_i = 1 \mid X_i).$$

Мы оцениваем вероятность успешного завершения модуля.

Что именно предсказываем

Целевая переменная бинарная:

$$Y_i = \begin{cases} 1, & \text{если модуль завершён,} \\ 0, & \text{если пользователь выбыл.} \end{cases}$$

То есть задача сводится к бинарной классификации.

При этом на практике важен не только сам класс, но и **уровень риска**. Поэтому модель должна выдавать вероятность:

$$\hat{p}_i \in [0, 1].$$

Чем меньше это значение, тем выше риск, что студент не дойдёт до конца модуля.

Такой формат прогноза удобен для ранжирования пользователей и для последующих управленческих действий.

Почему задача не совсем стандартная

Обучение в данных устроено по модулям:

$$M1 \to M2 \to M3.$$

Здесь возникает эффект отбора:

$$P(Y = 1 \mid M2) \gg P(Y = 1 \mid M1).$$

Это значит, что до модуля $M2$ доходят уже не все, а в среднем более устойчивые и вовлечённые пользователи.

Иначе говоря, во втором модуле мы видим не всю исходную популяцию, а уже "отфильтрованную" её часть. Это и есть **survivor bias**.

Чтобы модель учитывала этот сдвиг, в признаки добавляется номер модуля:

$$x_{\text{module}} \in \{1, 2, 3\}.$$

Так модель понимает, на каком этапе воронки находится пользователь.

Общая схема решения

```
.
├── data/
│   ├── raw/
│   │   └── *.csv
│   ├── preprocessing/
│   │   ├── final_user_course_features.csv
│   │   ├── M1_M4.csv
│   │   ├── df_train.parquet
│   │   ├── df_infer.parquet
│   │   └── figures/
│   └── eda_output/
│       ├── *.png
│       └── predictions.parquet
├── notebooks/
│   ├── preprocessing.ipynb
│   ├── leave_only_M1_M4.ipynb
│   ├── target_preparation.ipynb
│   ├── eda.py
│   └── old_preprocessing.ipynb
├── scripts/
└── requirements.txt
```

Схема показывает полную структуру проекта.

Схема запуска

1. `notebooks/preprocessing.ipynb`
   Загружает сырые таблицы из `data/raw`, нормализует ключи и типы, строит базовую сущность `users_course_id`, агрегирует feature-блоки по пользователю, курсу и `user-course`, объединяет их в master-таблицу и добавляет time-based признаки.
2. `notebooks/leave_only_M1_M4.ipynb`
   Читает `data/preprocessing/final_user_course_features.csv`, оставляет участников программы по наличию `stats_m1_row_count` / `stats_m2_row_count` / `stats_m3_row_count`, проставляет `module` и сохраняет результат в `data/preprocessing/M1_M4.csv`.
3. `notebooks/target_preparation.ipynb`
   Читает `M1_M4.csv`, строит `target` для `M1` и `M2` через `stats_m1_module_completed_flag` / `stats_m2_module_completed_flag`, формирует `hist_m1_*` и `hist_m2_*`, удаляет leakage-признаки и сохраняет `data/preprocessing/df_train.parquet` и `data/preprocessing/df_infer.parquet`.
4. `notebooks/eda.py`
   Читает `df_train.parquet` и `df_infer.parquet`, выполняет EDA по размеченной выборке, проверяет гипотезы, сравнивает модели через `StratifiedGroupKFold`, обучает лучшую модель и сохраняет inference в `data/eda_output/predictions.parquet` вместе с графиками.

`notebooks/old_preprocessing.ipynb` присутствует в репозитории, но по текущему коду не входит в основную цепочку: он отдельно читает raw-таблицы и не формирует артефакты, которые затем читают следующие шаги.

Какие данные использовались

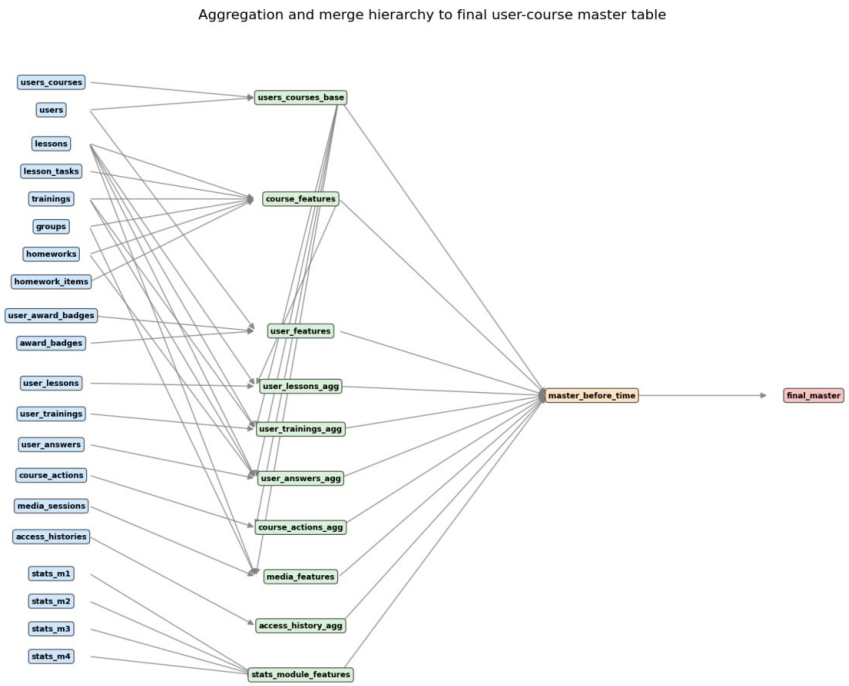
Данные в кейсе лежат не в одной таблице, а в наборе связанных сущностей. Основные источники:

- `users_courses` — пользователь на курсе;
- `users` — профиль пользователя;
- `lessons`, `lesson_tasks`, `groups` — структура курса;
- `user_lessons`, `user_trainings`, `user_answers` — учебное поведение;
- `wk_users_courses_actions` — действия внутри курса;
- `wk_media_view_sessions` — просмотры видео и трансляций;
- `stats_m1`–`stats_m4` — статусы и прогресс по модулям.

Поэтому первая часть проекта — это не обучение модели, а сборка корректной аналитической витрины на уровне

$$(user_id, course_id).$$

Схема сборки master-table



Master-table собирается не одним merge, а через несколько независимых блоков агрегации:

$$X_i = \bigcup_{k=1}^{K} f_k(\text{raw tables}).$$

Каждый блок берёт сырые таблицы и превращает их в компактный набор признаков нужной гранулярности.

Логика feature engineering

Весь feature engineering разбит на отдельные блоки:

- **users_courses_base** — базовая сущность;
- **course_features** — структура и сложность курса;
- **user_features** — свойства пользователя;
- **user_lessons** / **trainings** / **answers** — учебное поведение;
- **course_actions** / **media** — события и временные метрики;
- **stats_module** — таргет и история модулей.

Такой подход удобен по двум причинам:

- каждый блок можно разрабатывать и проверять отдельно;
- проще контролировать merge, пропуски и leakage.

Как устроен итоговый вектор признаков

Итоговый вектор признаков можно записать так:

$$X_i = \left[X_i^{\text{course}}, X_i^{\text{user}}, X_i^{\text{lesson}}, X_i^{\text{training}}, X_i^{\text{answers}}, X_i^{\text{actions}}, X_i^{\text{media}}\right].$$

То есть итоговая модель опирается не на один тип сигналов, а сразу на несколько:

- структуру курса;
- свойства пользователя;
- поведение в уроках и тренингах;
- поток действий и медиа-активность.

Именно такое разложение позволяет отделить **контекст курса** от **реального поведения студента**.

Базовая сущность: users_courses_base

В основе master-table лежит таблица `users_courses`. Она задаёт основную гранулярность: один пользователь на одном курсе.

Примеры признаков:

$$\text{uc_points_ratio} = \frac{\text{wk_points}}{\text{wk_max_points}}$$

$$\text{uc_tasks_per_viewable_lesson} = \frac{\text{wk_max_task_count}}{\text{wk_max_viewable_lessons}}.$$

Первая величина показывает, какую долю от максимума пользователь уже набрал. Вторая — какую учебную нагрузку в среднем несёт один доступный урок.

Также timestamp переводятся в числовые признаки в днях:

$$\text{uc_enrollment_age_days} = t_{\text{ref}} - t_{\text{created}},$$

$$\text{uc_official_start_delay_days} = t_{\text{official start}} - t_{\text{created}}.$$

Course-level признаки

Эти признаки отвечают на вопрос: *насколько сам курс тяжёлый и насыщенный*

Примеры:

$$\text{course_tasks_per_lesson} = \frac{\text{course_lessons_task_count_sum}}{\text{course_lessons_count}},$$

$$\text{course_points_per_lesson} = \frac{\text{course_lessons_max_points_sum}}{\text{course_lessons_count}},$$

$$\text{course_video_duration_per_lesson} = \frac{\text{course_video_duration_sum}}{\text{course_lessons_count}}.$$

Чем больше задач и баллов приходится на урок, тем выше учебная нагрузка.
Чем длиннее видео, тем выше порог входа по времени.

Эти признаки важны, потому что поведение студента нужно оценивать с поправкой на сложность самого курса.

User-level признаки

Пользовательские признаки не зависят от конкретного урока. Они описывают общий профиль и устойчивость аккаунта.

Примеры:

$$\text{user_account_age_days} = t_{\text{ref}} - t_{\text{user created}},$$

$$\text{user_badges_per_account_year} = \frac{\text{user_badges_total_count}}{\text{user_account_age_days}/365.25}.$$

Первый признак показывает, как давно пользователь находится в системе. Второй — насколько активно он в среднем взаимодействует с платформой в долгом горизонте.

Сами по себе такие признаки редко решают всё, но они хорошо помогают сегментировать пользователей.

Поведенческие признаки: действия и медиа

Самые полезные признаки в итоговой модели связаны с темпом активности.

Ключевые признаки:

$$x_1 = \text{time_training_intensity_per_observed_day},$$

$$x_2 = \text{time_media_intensity_per_observed_day},$$

$$x_3 = \text{time_to_first_action_days}.$$

Идея первых двух признаков такая:

$$x_1 = \frac{\text{число учебных действий}}{\text{число наблюдаемых дней}}, \qquad x_2 = \frac{\text{число медиа-действий}}{\text{число наблюдаемых дней}}.$$

Мы используем именно интенсивность, а не абсолютное число событий, чтобы убрать эффект длины наблюдения.

Признак x_3 показывает, насколько быстро пользователь вообще включился в работу после входа в курс.

Как формировался таргет

Таргет строится отдельно для разных модулей:

$$Y = \begin{cases} \text{stats_m1_module_completed_flag}, & \text{для } M1, \\ \text{stats_m2_module_completed_flag}, & \text{для } M2, \\ \text{NULL}, & \text{для } M3. \end{cases}$$

То есть:

- для $M1$ и $M2$ исход уже известен;
- для $M3$ исход на момент построения модели ещё неизвестен.

Поэтому разбиение выглядит так:

$$\mathcal{D}_{train} = \{M1, M2\}, \qquad \mathcal{D}_{infer} = \{M3\}.$$

Это важно: модель проверяется не на случайном holdout, а на естественном сценарии будущего применения.

Почему leakage здесь особенно опасен

Если в модель случайно попадают признаки, которые уже содержат информаци
об исходе, качество окажется искусственно завышенным.

Формально leakage возникает, если

$$x_j = f(Y, t > t_0),$$

то есть признак использует данные из будущего относительно момента прогноза.
Тогда получается техническая связь

$$x_j \to Y,$$

которая не отражает реальную предсказательную силу.

Поэтому корректное требование такое:

$$X \perp \text{future outcome information}.$$

Проще говоря, модель должна видеть только прошлое и текущее поведение, но не подсматривать в финальный результат.

Hard leakage

Hard leakage — это признаки, которые почти напрямую кодируют ответ. Например:

- `wk_course_completed_at`
- `uc_has_completion_record`
- `uc_completion_delay_days`
- `uc_completed_flag`
- `uc_completed_without_points_flag`

Если такие признаки оставить, модель будет не прогнозировать исход, а фактически считывать его из данных.

Поэтому все такие столбцы удалялись до обучения.

Soft leakage

Soft leakage — это признаки, которые формально не совпадают с таргетом, но слишком близки к нему по смыслу.

В эту группу попадают, например:

- `*_row_count`
- `*_lessons_viewed_count`
- `*_content_viewed_units`
- `*_final_tasks_solved_count`
- `*_interim_assessment_score`

Такие признаки вроде бы не содержат прямой метки завершения, но на практике оказываются слишком близки к итоговому статусу пользователя.

Исторические признаки

Отдельно контролировались исторические признаки по предыдущим модулям.

Чтобы не использовать информацию из будущего, применялись такие правила:

- `hist_m1_*` зануляются для строк $M1$;
- `hist_m2_*` зануляются для строк $M1$ и $M2$.

Почему это важно: история модуля может использоваться только там, где этот модуль уже действительно произошёл во времени.
Иначе модель начнёт получать информацию, которой в момент прогноза ещё не должно быть.

Почему финальная модель компактная

После feature engineering признаков получилось много, но в итоговую модель вошёл компактный набор:

$$X^{final} = \{x_1, x_2, x_3, x_4, x_5\},$$

где

$$x_1 = \text{training intensity/day}, \quad x_2 = \text{media intensity/day},$$

$$x_3 = \text{time to first action}, \quad x_4 = \text{module_num}, \quad x_5 = \text{course_id}.$$

Это не упрощение ради упрощения. Это попытка оставить самые устойчивые, интерпретируемые и безопасные признаки.

Такой набор лучше переносится на новые данные и проще объясняется на защите.

Статистическая проверка признаков

Перед обучением мы отдельно проверяли, различают ли признаки completed и dropout.
Нулевая гипотеза:

$$H_0 : X_j \perp Y.$$

То есть признак X_j не связан с целевой переменной.

Для числовых признаков использовался критерий Манна–Уитни:

$$U = \min(U_1, U_2).$$

Решение принималось по p-value:

$$p < 0.05 \Rightarrow H_0 \text{ отвергается.}$$

Этот тест удобен здесь, потому что он не требует нормальности распределения и устойчив к выбросам.

Point-biserial correlation

Дополнительно признаки ранжировались по point-biserial корреляции:

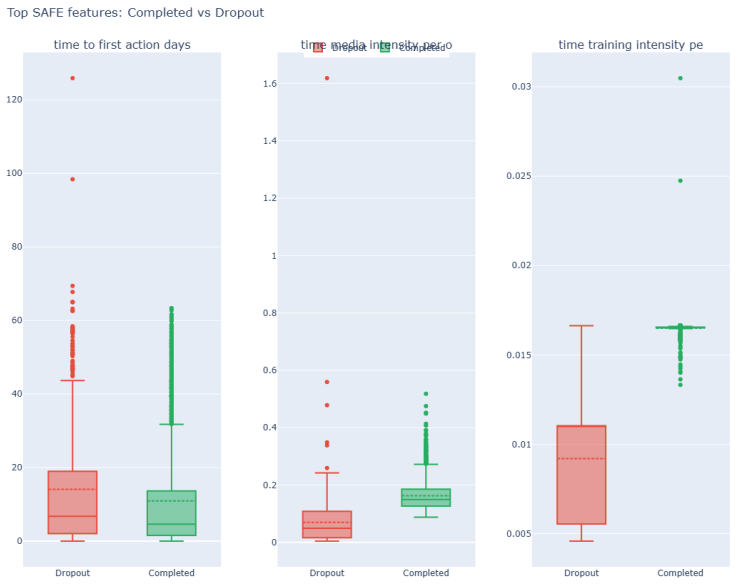
$$r_{pb} = \frac{\mu_1 - \mu_0}{\sigma}\sqrt{pq}.$$

Здесь:

- μ_1 — среднее значение признака у completed;
- μ_0 — среднее значение признака у dropout;
- σ — общее стандартное отклонение;
- p и q — доли двух классов.

Смысл формулы такой: чем сильнее отличаются средние двух классов относительно общего разброса, тем полезнее признак для разделения completed и dropout.

Распределение ключевых признаков

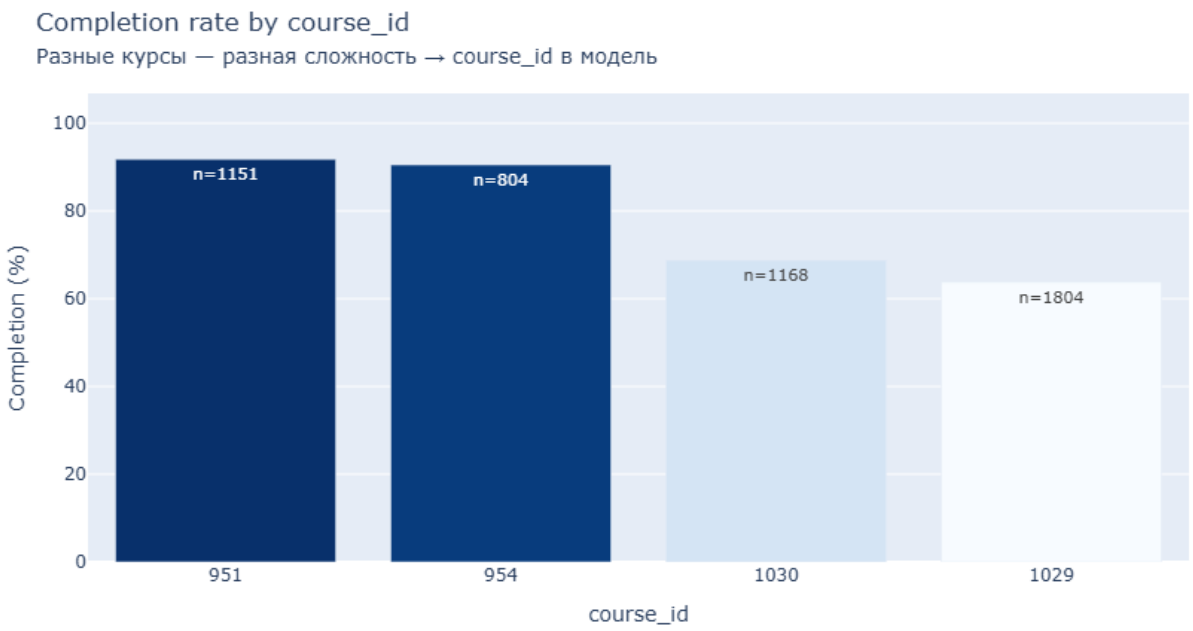


На графике показаны распределения отобранных признаков для двух классов: завершившие модуль (зелёный) и выбывшие (красный).
Можно заметить, что пользователи, успешно завершившие модуль:

- быстрее начинали работу в курсе (меньшее время до первого действия);
- проявляли более высокую активность в процессе обучения.

Это подтверждает гипотезу о важности раннего вовлечения.

Различия между курсами

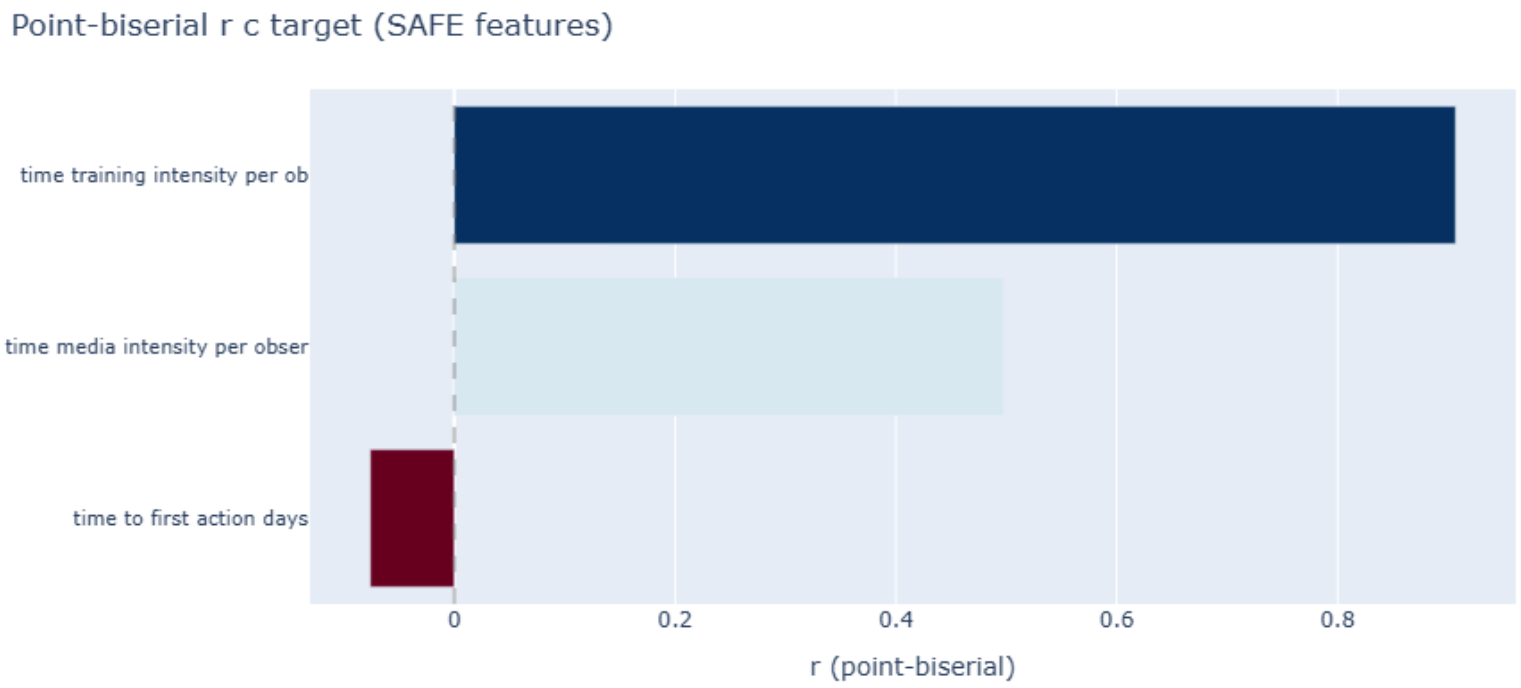


Для разных курсов наблюдается различный уровень успешного завершения.

Это означает, что вероятность завершения зависит не только от поведения пользователя, но и от характеристик самого курса.

В связи с этим идентификатор курса используется в модели как категориальный признак, позволяющий учитывать различия между курсами.

Связь признаков с целевой переменной

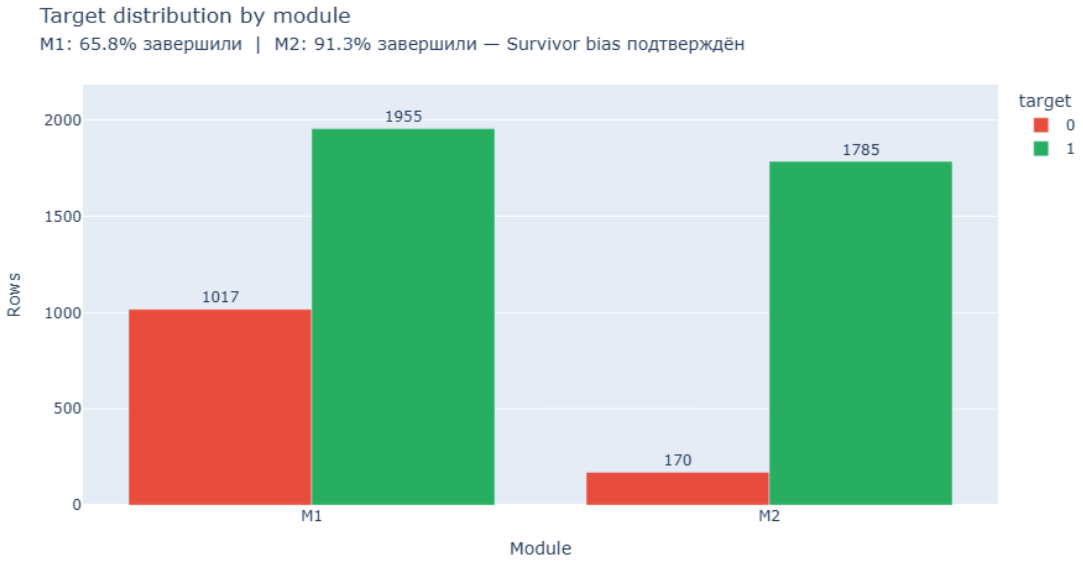


На графике представлены корреляции между целевой переменной (завершение модуля) и отобранными признаками.

Видно, что ряд признаков демонстрирует заметную связь с таргетом, что подтверждает их информативность для задачи классификации.

Данный анализ использовался как дополнительный этап отбора признаков перед обучением модели.

Роль номера модуля



Номер модуля был включён в модель как отдельный признак.
Это связано с тем, что пользователи, перешедшие к более поздним модулям, уже прошли предварительный отбор:

- они в среднем более вовлечены;
- имеют меньшую вероятность досрочного выхода.

Таким образом, номер модуля отражает положение пользователя в воронке обучения.

Как была устроена валидация

Для оценки качества использовался

$$\text{StratifiedGroupKFold}.$$

Это означает сразу две вещи:

1. баланс классов сохраняется по фолдам;
2. все строки одного пользователя попадают только в один фолд.

Формально:

$$\forall u \in \mathcal{U} \quad \exists!\, k : \text{all rows of user } u \in \text{fold } k.$$

Это критично, потому что один и тот же пользователь может встречаться в нескольких модулях. Без группировки качество было бы завышено.

Модель 1: Logistic Regression

Логистическая регрессия оценивает вероятность класса так:

$$P(Y = 1 \mid X) = \sigma(w^\top X + b),$$

где

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Здесь $w^\top X + b$ — линейный скор, а сигмоида переводит его в число от 0 до 1.

Плюсы этой модели в нашем случае:

- она хорошо интерпретируется;
- подходит для компактного набора признаков;
- задаёт сильный и честный baseline.

Модель 2: Random Forest

Random Forest строит ансамбль деревьев решений:

$$\hat{y}(X) = \frac{1}{T} \sum_{t=1}^{T} f_t(X),$$

где T — число деревьев, а $f_t(X)$ — прогноз отдельного дерева.

Идея в том, что каждое дерево видит данные по-своему, а итоговый прогноз получается усреднением.

Почему эта модель была полезна:

- умеет ловить нелинейные зависимости;
- нормально работает на смешанных признаках;
- устойчива после импутации пропусков.

Что оптимизировалось

Модели сравнивались по ROC-AUC:

$$\mathrm{AUC} = P(\hat{s}(X^+) > \hat{s}(X^-)),$$

где X^+ — объект положительного класса, X^- — отрицательного.

Иными словами, AUC — это вероятность того, что модель поставит completed выше dropout.

Эта метрика хорошо подходит здесь, потому что:

- не зависит от конкретного порога;
- устойчива при дисбалансе классов;
- отражает качество ранжирования риска.

Инференс и практический смысл

После выбора лучшей модели она дообучается на всём наборе

$$\mathcal{D}_{train} = \{M1, M2\},$$

а затем выдаёт прогнозы на

$$\mathcal{D}_{infer} = \{M3\}.$$

Для каждого пользователя получаем

$$\hat{p}_i = P(Y_i = 1 \mid X_i).$$

Дальше эти вероятности можно использовать практически:

- ранжировать студентов по риску;
- выделять группу для раннего вмешательства;
- приоритизировать работу кураторов;
- сравнивать курсы и поведенческие паттерны.

Главные выводы

1. Самый сильный сигнал лежит в раннем поведении и темпе активности.
2. Сырые stats-агрегаты без контроля leakage использовать нельзя.
3. Group-aware валидация здесь обязательна.
4. Компактная и понятная модель в этой задаче лучше перегруженной.

Результаты: Logistic Regression

Качество модели оценивалось с помощью кросс-валидации.

Среднее значение ROC-AUC:

$$\text{mean} = 0.8945 \pm 0.0109$$

Значения по фолдам:

- Fold 1: 0.8814
- Fold 2: 0.9021
- Fold 3: 0.9016
- Fold 4: 0.8813
- Fold 5: 0.9062

Интерпретация: модель показывает стабильное качество без сильных провалов между фолдами.

Итог

В проекте получился полный ML pipeline:

- сборка master-table из нескольких источников;
- feature engineering;
- построение таргета;
- удаление hard и soft leakage;
- статистическая проверка признаков;
- корректная валидация;
- обучение и inference.